

migration_002_design

Helix OTA — Migration 002 Design (Staged Rollout: deployment_phases, rollouts, rollback_history)

Field	Value
Revision	1
Created	2026-06-08
Last modified	2026-06-08
Status	active (design — DDL to be authored + validated against live Postgres before apply)
Status summary	Design of migration 002_staged_rollout that extends the helix_ota schema (migration 001) with the three tables deferred from MVP: deployment_phases (the ordered percentage plan), rollouts (the per-deployment engine cursor + control-plane status), and rollback_history (audit of abort/rollback/recall). Backs the storage port in rollout_engine.md §6. TUF metadata is explicitly deferred (§5). This is a design document: it specifies tables, columns, constraints, indexes, the deployments/rollouts integration, the up/down pair, and the validation gate. The literal .up.sql/.down.sql are authored from this design and validated against live Postgres (as migration 001 was), then placed in 1.0.0-mvp/database/migrations/ — UNVERIFIED whether 1.0.1 migrations co-locate there or move to a 1.0.1-staged-rollout/database/migrations/ dir (§6.4).
Status summary 2	

Field	Value
Issues	The brick treats Phases as immutable; the schema enforces that with no UPDATE path on deployment_phases after rollout start (§3.1). Two status columns on rollouts (engine vs control-plane, rollout_engine.md §6.3) must stay reconciled by a single writer — encoded as constraints + app discipline, not a DB trigger (§3.2).
Fixed	Promotes README.md §6 (“adds deployment_phases, rollouts, rollback_history ... migration 002_*) into concrete DDL design aligned to migration 001’s conventions (schema helix_ota, gen_random_uuid(), TIMESTAMPTZ, CHECK-constrained enums, BEGIN/COMMIT transactional).
Continuation	Author 002_staged_rollout.up.sql + .down.sql from this design; validate up→down→up against a live Postgres 14+; wire the storage-port repo (rollout_engine.md §6); add migration-runner test.
Owner	Helix OTA control-plane team
Related	rollout_engine.md; ../1.0.0-mvp/database/migrations/001_initial_schema.up.sql; ../1.0.0-mvp/database/migrations/001_initial_schema.down.sql; README.md; ../research/additions_synthesis.md (§8 G7, §10 K8); submodules/ota-rollout-engine

table of contents

- §1. purpose and scope
- §2. conventions inherited from migration 001
- §3. table designs
 - §3.1 deployment_phases
 - §3.2 rollouts
 - §3.3 rollback_history
- §4. deployments integration (no breaking change)
- §5. tuf metadata note (deferred — not created)
- §6. up/down migration design and validation
- §7. mapping to the engine state (storage port)
- §8. anti-bluff / unverified register

§1. purpose and scope

Migration 001 deliberately **omitted** staged-rollout and rollback tables, documenting them as “DEFERRED to 1.0.1” (001 header lines 8–11). Migration 002 adds exactly those, no more:

- `deployment_phases` — the ordered percentage plan for a staged deployment; one row per phase, mirroring the brick’s `Phase` struct (`rollout_engine.md` §2.1).
- `rollouts` — one row per staged deployment: the brick engine cursor (`current_phase`, `engine_status`, `phase_started_at`) plus the control-plane lifecycle status (`paused/aborted/rolled_back`). This is the row the `StoragePort` loads/saves (`rollout_engine.md` §6.2).
- `rollback_history` — append-only audit of every abort / server-driven recall (rollback), closing synthesis §8 G7’s `rollback_history` item and feeding 1.0.2’s end-user rollback.

Out of scope (explicitly **not** created): any TUF metadata table (§5), any column on deployments (the MVP `rollout_strategy` JSONB already carries the staged marker, §4), and end-user/multi-version rollback state (1.0.2, synthesis §10 K8).

§2. conventions inherited from migration 001

Migration 002 follows 001 byte-for-byte in style so the corpus stays uniform:

- Wrapped in a single `BEGIN; ... COMMIT;` (all-or-nothing on transactional runners; 001 note).
- `SET LOCAL search_path = helix_ota, public;`
- PKs are `UUID PRIMARY KEY DEFAULT gen_random_uuid()` (`pgcrypto`, already created by 001).
- Timestamps `TIMESTAMPTZ NOT NULL DEFAULT now()`.
- Enums as `VARCHAR(n) + CHECK (... IN (...))` (not native PG enums — matches 001’s `*_status_chk` pattern, keeps additive evolution cheap).
- FKs named `<table>_<ref>_fk` with explicit `ON DELETE` semantics; indexes named `idx_<table>_<cols>`.
- JSONB for open/extensible blobs (metadata, details) defaulting to `'{}'::jsonb`.
- `helix_ota` schema only (modular-monolith, ADR-0003 — one schema; 001 line 16).

§3. table designs

§3.1 `deployment_phases`

One row per ordered phase of a staged deployment. Columns mirror the brick `Phase` (`rollout_engine.md` §2.1) so a Load materializes `[]Phase` directly.

Column	Type	Notes
<code>id</code>	UUID PK	<code>gen_random_uuid()</code>

Column	Type	Notes
deployment_id	UUID NOT NULL	FK → deployments(id) ON DELETE CASCADE
phase_index	INT NOT NULL	0-based order; the brick's slice index
percentage	INT NOT NULL	cumulative cohort %, the brick Phase.Percentage
success_threshold	NUMERIC(4,3) NOT NULL	fraction [0,1] (e.g. 0.950)
error_threshold	NUMERIC(4,3) NOT NULL	fraction [0,1]
duration_seconds	BIGINT NOT NULL	evaluation window in seconds; 0 = no time bound (brick: Duration==0)
auto_progress	BOOLEAN NOT NULL DEFAULT TRUE	brick Phase.AutoProgress
created_at	TIMESTAMP TZ NOT NULL DEFAULT now()	

Constraints (enforce the brick's validatePhases at the DB layer too — defense in depth):

```

CONSTRAINT deployment_phases_deployment_fk
    FOREIGN KEY (deployment_id) REFERENCES helix_ota.deployments
(id) ON DELETE CASCADE,
CONSTRAINT deployment_phases_index_chk    CHECK (phase_index >=
0),
CONSTRAINT deployment_phases_pct_chk      CHECK (percentage > 0
AND percentage <= 100),
CONSTRAINT deployment_phases_success_chk  CHECK
(success_threshold >= 0 AND success_threshold <= 1),
CONSTRAINT deployment_phases_error_chk    CHECK
(error_threshold >= 0 AND error_threshold <= 1),
CONSTRAINT deployment_phases_duration_chk CHECK
(duration_seconds >= 0),
CONSTRAINT deployment_phases_idx_uniq     UNIQUE (deployment_id,
phase_index),
CONSTRAINT deployment_phases_pct_uniq     UNIQUE (deployment_id,
percentage)

```

Index: idx_deployment_phases_deployment ON deployment_phases
(deployment_id, phase_index) (covers the ordered Load).

Notes:

- The **strictly-increasing** and **final-phase== 100** invariants are cross-row and are NOT expressed as table CHECKs (Postgres CHECK is per-row). They are enforced by the brick's validatePhases at create time (rollout_engine.md

§8.2) and re-asserted by a migration-time / repo-level assertion; the deployment_phases_pct_uniq constraint at least forbids duplicate percentages. (UNVERIFIED whether to add an EXCLUSION/trigger for strict monotonicity — deferred; the brick is the authoritative validator, §8.)

- **Immutability:** the brick treats Phases as immutable. The repo writes these rows once (at POST .../rollout, rollout_engine.md §8.2) and never UPDATES them; Save touches only rollouts. No DB trigger enforces this in 002 (app discipline + no UPDATE path); a future hardening could add an AFTER UPDATE guard.

§3.2 rollouts

One row per staged deployment — the StoragePort key (rollout_engine.md §6.2). Carries **both** status columns (rollout_engine.md §6.3).

Column	Type	Notes
id	UUID PK	
deployment_id	UUID NOT NULL	FK → deployments(id) ON DELETE CASCADE; UNIQUE (one rollout per deployment) — also the storage-port key
engine_status	VARCHAR(20) NOT NULL DEFAULT 'pending'	brick Status: pending/ active/halted/held/ completed
status	VARCHAR(20) NOT NULL DEFAULT 'pending'	control-plane status: brick set + paused/aborted/ rolled_back
current_phase	INT NOT NULL DEFAULT 0	brick State.CurrentPhase (index into deployment_phases)
phase_started_at	TIMESTAMPTZ	brick State.PhaseStartedAt; NULL before start
started_at	TIMESTAMPTZ	when the rollout was first started
completed_at	TIMESTAMPTZ	terminal timestamp (completed/aborted/ rolled_back)
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()	
updated_at	TIMESTAMPTZ NOT NULL DEFAULT now()	brick State.UpdatedAt; bumped on every Save

Constraints:

```
CONSTRAINT rollouts_deployment_fk
    FOREIGN KEY (deployment_id) REFERENCES helix_ota.deployments
```

```
(id) ON DELETE CASCADE,
CONSTRAINT rollouts_deployment_uniq UNIQUE (deployment_id),
CONSTRAINT rollouts_engine_status_chk
    CHECK (engine_status IN
('pending','active','halted','held','completed')),
CONSTRAINT rollouts_status_chk
    CHECK (status IN
('pending','active','halted','held','completed','paused','aborted'),
CONSTRAINT rollouts_current_phase_chk CHECK (current_phase >= 0),
CONSTRAINT rollouts_time_order_chk
    CHECK (completed_at IS NULL OR started_at IS NULL OR
completed_at >= started_at)
```

Indexes:

```
idx_rollouts_status ON rollouts (status)          -- scheduler
scans active, non-paused
idx_rollouts_engine_status ON rollouts (engine_status)
```

Reconciliation rule (app-enforced, §2 conventions — no trigger): the control plane is the single writer (rollout_engine.md §6.3). When `status` $\in$ {active, held, halted, completed, pending} it MUST equal `engine_status`; paused/aborted/rolled_back are control-plane-only overlays. (UNVERIFIED whether to harden this with a CHECK relating the two columns — a partial CHECK like “`status = engine_status OR status IN ('paused', 'aborted', 'rolled_back')`” is feasible and recommended; deferred to DDL authoring, §8.)

§3.3 rollback_history

Append-only audit of abort / server-driven recall (rollback). Closes synthesis §8 G7’s `rollback_history` item. FKs use `ON DELETE SET NULL` so history outlives the deployments it references (mirrors 001’s `telemetry_events/audit_logs` survivability pattern).

Column	Type	Notes
id	UUID PK	
deployment_id	UUID	FK → <code>deployments(id)</code> ON DELETE SET NULL — the rollout that was aborted/recalled
kind	VARCHAR(20) NOT NULL	abort (halt forward progress) or rollback (server-driven recall to N-1)
from_release_id	UUID	FK → <code>releases(id)</code> ON DELETE SET NULL — release being rolled away from (NULL for abort)
to_release_id	UUID	

Column	Type	Notes
		FK → releases(id) ON DELETE SET NULL — previous-good release recalled to (NULL for abort)
recall_deployment_id	UUID	FK → deployments(id) ON DELETE SET NULL — the new deployment created by a recall (rollout_engine.md §5.6)
reason	TEXT	operator-supplied reason / engine halt reason
triggered_by	UUID	FK → users(id) ON DELETE SET NULL (NULL when engine/ scheduler-triggered)
details	JSONB NOT NULL DEFAULT '{}::jsonb	brick Decision.Reason, cohort %, counts, etc.
created_at	TIMESTAMPTZ NOT NULL DEFAULT now()	

Constraints:

```

CONSTRAINT rollback_history_kind_chk CHECK (kind IN
('abort','rollback')),
CONSTRAINT rollback_history_deployment_fk
    FOREIGN KEY (deployment_id) REFERENCES
helix_ota.deployments (id) ON DELETE SET NULL,
CONSTRAINT rollback_history_from_release_fk
    FOREIGN KEY (from_release_id) REFERENCES
helix_ota.releases (id) ON DELETE SET NULL,
CONSTRAINT rollback_history_to_release_fk
    FOREIGN KEY (to_release_id) REFERENCES
helix_ota.releases (id) ON DELETE SET NULL,
CONSTRAINT rollback_history_recall_fk
    FOREIGN KEY (recall_deployment_id) REFERENCES
helix_ota.deployments (id) ON DELETE SET NULL,
CONSTRAINT rollback_history_user_fk
    FOREIGN KEY (triggered_by) REFERENCES helix_ota.users
(id) ON DELETE SET NULL,
-- A 'rollback' records both from/to releases; an 'abort' records
neither.
CONSTRAINT rollback_history_kind_ref_chk CHECK (
    (kind = 'rollback' AND from_release_id IS NOT NULL AND
to_release_id IS NOT NULL)
    OR (kind = 'abort' AND from_release_id IS NULL AND
to_release_id IS NULL)
)

```

Indexes:

```
idx_rollback_history_deployment ON rollback_history
(deployment_id)
idx_rollback_history_kind        ON rollback_history (kind)
idx_rollback_history_created    ON rollback_history (created_at)
```

The `kind_ref_chk` is a design choice; if 1.0.2 introduces rollback kinds where releases are unknown, relax it then. For 1.0.1 the two kinds have exactly these reference shapes (`rollout_engine.md §5.5/§5.6`).

§4. deployments integration (no breaking change)

Migration 002 adds **no column** to deployments. The MVP `deployments.rollout_strategy JSONB` (001 lines 252) already carries the strategy marker; a staged deployment writes:

```
{ "mode": "staged" }
```

instead of the MVP `{"mode": "all_at_once"}`. The MVP `deployments.status CHECK` already includes `paused` (001 line 270: `draft/active/paused/completed/failed/cancelled`), so `pause` maps cleanly onto the existing deployment status without a 002 alteration. `device_deployments.phase_id` was intentionally omitted at MVP (001 line 289); 002 does **not** add it either — `phase` membership is computed deterministically from `(device_id, deployment_id, current_percentage)` via the brick's `InCohort` (`rollout_engine.md §4`), so it need not be persisted per device. (UNVERIFIED future need: if per-device phase attribution is required for analytics, add a nullable `device_deployments.phase_index` in a later migration — not now, to avoid a write per device.)

This keeps 002 **purely additive** (three new tables, zero `ALTER TABLE`), the safest migration shape and easiest to reverse (§6).

§5. tuf metadata note (deferred — not created)

Per `README.md §6` (“plus TUF metadata storage”) and `ADR-0002 / synthesis §10 C5/K2`, device-side TUF/Uptane is **deferred**. Migration 002 **does NOT create any TUF metadata table**. This is recorded explicitly (as 001 recorded the staged-rollout deferral) so the omission is a documented decision, not an oversight:

- The 1.0.1 trust model stays MVP plain-signing (`ed25519 + SHA-256 + AVB`), layered so TUF drops in byte-identically later (`endpoints.md §1`; `rollout_engine.md §11`).
- When TUF lands (later 1.0.1 sub-spec or 1.0.2), a separate migration (e.g. `003_tuf_metadata`) will add `tuf_metadata` (`root/timestamp/snapshot/targets` roles, version, expiry) and the Director/Image repository split — **independent** of the rollout tables. The rollout engine is trust-model-agnostic: adding TUF MUST NOT alter `deployment_phases/rollouts/rollback_history`.

The DDL header comment of `002_*.up.sql` MUST carry this deferral note verbatim (matching 001's in-file deferral comments).

§6. up/down migration design and validation

§6.1 up (`002_staged_rollout.up.sql`)

BEGIN; → SET LOCAL search_path → CREATE TABLE deployment_phases → CREATE TABLE rollouts → CREATE TABLE rollback_history → the indexes → COMMIT;. Creation order respects FKs (deployment_phases/rollouts/rollback_history all reference 001 tables that already exist; no inter-002 FK ordering hazard). No CREATE EXTENSION/CREATE SCHEMA needed (001 made them).

§6.2 down (`002_staged_rollout.down.sql`)

Reverse order, DROP TABLE IF EXISTS ... CASCADE for rollback_history, then rollouts, then deployment_phases, inside BEGIN;...COMMIT;. Because 002 is purely additive (no ALTER), the down is a clean drop with no data-restoration concern; deployments.rollout_strategy values of {"mode":"staged"} written while 002 was live are left as-is (harmless JSONB the MVP code treats as an unknown mode → it MUST fall back to not offering staged behavior; UNVERIFIED that MVP code defensively handles an unknown mode, §8).

§6.3 validation gate (as migration 001)

Before the DDL is accepted it MUST pass, against a **live Postgres 14+** (not just lint):

1. up applies cleanly on a DB with 001 already applied.
2. down reverses cleanly (tables gone, 001 intact).
3. up → down → up is idempotent (re-apply works).
4. The brick's validatePhases boundary cases round-trip: a valid 5/10/30/50/100 plan inserts; a duplicate-percentage plan is rejected by deployment_phases_pct_uniq; out-of-range threshold rejected by the CHECKs.
5. A migration-runner test asserts the three tables + their constraints exist (artifact gate, rollout_engine.md §10 layer 2).

§6.4 file placement (UNVERIFIED)

Migration 001 lives in `1.0.0-mvp/database/migrations/`. Whether 002 co-locates there (single linear migration history for the one helix_ota schema — recommended, since migrations are schema-global not phase-scoped) or under `1.0.1-staged-rollout/database/migrations/` is an **UNVERIFIED** corpus-layout decision (§8). This design recommends **co-location** in `1.0.0-mvp/database/migrations/002_staged_rollout.{up,down}.sql` so the runner sees one ordered sequence; the spec lives here in the 1.0.1 phase dir.

§7. mapping to the engine state (storage port)

The `StoragePort` impl (`rollout_engine.md` §6.2) materializes `rollout.State` from these tables:

<code>rollout.State</code>	Source
<code>DeploymentID</code>	<code>rollouts.deployment_id</code>
<code>Phases</code>	<code>SELECT ... FROM deployment_phases</code> <code>WHERE deployment_id=\$1 ORDER BY</code> <code>phase_index, each row →</code> <code>Phase{Percentage,</code> <code>SuccessThreshold,</code> <code>ErrorThreshold, Duration:</code> <code>duration_seconds*time.Second,</code> <code>AutoProgress}</code>
<code>CurrentPhase</code>	<code>rollouts.current_phase</code>
<code>Status</code>	<code>rollouts.engine_status</code>
<code>PhaseStartedAt</code>	<code>rollouts.phase_started_at</code>
<code>UpdatedAt</code>	<code>rollouts.updated_at</code>

Save upserts only the rollouts cursor columns (`engine_status`, `current_phase`, `phase_started_at`, `updated_at`) under `SELECT ... FOR UPDATE` on the rollouts row (`rollout_engine.md` §6.4); `deployment_phases` is never updated by `Save`. `Load` returns `rollout.ErrNotFound` (wrapped) when no rollouts row exists for the deployment id.

The control-plane lifecycle verbs (`rollout_engine.md` §5) write `rollouts.status` (and `rollback_history`) directly, outside the brick's `Save` path, keeping the brick's engine cursor and the control-plane overlay in distinct columns (§3.2).

§8. anti-bluff / unverified register

Per Constitution §11.4.6 (no-guessing) / §7.1 (no-bluff); **MUST NOT** be propagated as fact.

- **This is a design, not the applied DDL.** The literal `.up.sql/.down.sql` are authored from this design and **validated against live Postgres** (§6.3) before being trusted — exactly as migration 001 was. No claim here is “it runs” until that gate passes.
- **Cross-row phase invariants** (strictly-increasing percentage, final = 100) are **not** expressible as per-row Postgres CHECKs; they are enforced by the brick's `validatePhases` (verified in `types.go`), with `deployment_phases_pct_uniq` as the only DB-level guard against duplicates. Whether to add an EXCLUSION constraint / trigger for strict monotonicity is **UNVERIFIED / deferred** (§3.1).
- **The two-status reconciliation** (§3.2) is enforced by app discipline (single writer); a relating CHECK is **recommended but UNVERIFIED** pending DDL authoring.

- **deployment_phases immutability** has **no DB trigger** in 002 — relies on the repo never issuing an UPDATE (the brick treats Phases as immutable, verified). A future AFTER UPDATE guard is possible.
- **MVP code's handling of an unknown rollout_strategy.mode** after a 002 down-migration (§6.2) is **UNVERIFIED** — it should defensively treat any non-all_at_once mode as non-staged.
- **File placement** of 002 (§6.4) is an **UNVERIFIED** corpus-layout decision; co-location with 001 is recommended, not confirmed.
- **device_deployments.phase_index** is intentionally NOT added; the future need for per-device phase attribution is **UNVERIFIED** (§4).
- **Numeric precision choices** (NUMERIC(4,3) for thresholds, BIGINT seconds for duration) are design picks sufficient for [0,1] fractions and time.Duration ranges; confirm against the brick's float64/time.Duration at repo-mapping time.
- **HelixConstitution clause numbers** cited remain **UNVERIFIED** against the authoritative text (consistent with endpoints.md §16 and migration 001's provenance notes).